

Lab 3 Instructions

Aditya Sarkar

Tulane Dr, Hyattsville, MD | September 23, 2026

Problem 1: Synchronized Benchmark Loop.

Write a function called `run_timed_iterations` that measures how long a workload takes to run across multiple iterations. Steps for implementing it are as follows:

- Call `bench.workload.synchronize()` once before starting the loop to ensure no previous work is pending.
- Loop `repeats` times. In each loop iteration:
 - Record the start time using `bench.clock()`.
 - Execute the workload by calling `bench.workload.run()`.
 - Call `bench.workload.synchronize()` to wait for all asynchronous GPU execution to finish.
 - Record the end time using `bench.clock()`.
 - Calculate the elapsed duration, convert it from nanoseconds to milliseconds by dividing by 1,000,000.0, and store it in a list.
- Return the list of elapsed times in milliseconds.

Problem 2: Warm-up Detection (`find_warmup_boundary`)

Write a function called `find_warmup_boundary` that detects and removes the initial iterations where the system was warming up. Steps for implementing it are as follows:

- Check the length of `samples`. If there are fewer than 4 items, return an `unknown` record stating that there are too few samples.
- Take the second half of the sample list and calculate its median value using `statistics.median`. This represents the settled steady-state time. If this median is ≤ 0 , return an `unknown` record.
- Calculate the cutoff threshold as: $\text{threshold} = \text{median} \times (1 + \text{WARMUP_TOL})$, where $\text{WARMUP_TOL} = 0.5$.
- Count how many consecutive samples from the start of the list are strictly greater than this threshold.

- Return a **measured** record storing the count of discarded warm-up iterations, along with the settled median, the threshold, the tolerance value, and the number of retained iterations.

Problem 3: Statistical Summary (`summarize`)

Write a function called `summarize(samples)` that computes summary statistics for a list of timing measurements. Steps for implementing it are as follows:

- If `samples` is empty, return a dictionary with `"n": 0` and set all metric keys (`"mean"`, `"std"`, `"min"`, `"max"`, `"p50"`, `"p95"`, `"p99"`) to `None`.
- Sort the samples in ascending order.
- Compute the mean using `statistics.fmean`, the minimum value, and the maximum value.
- Compute the standard deviation using `statistics.stdev` if there are more than 2 samples; otherwise set it to 0.0.
- Compute the 50th, 95th, and 99th percentiles using linear interpolation between the sorted values:

$$h = (n - 1) \times \frac{q}{100}, \quad i = \lfloor h \rfloor, \quad \text{value} = s[i] + (h - i) \times (s[i + 1] - s[i])$$

- Round all floating-point numbers to 4 decimal places and return them in a dictionary.

Problem 4: Multimodal Distribution Check (`is_multimodal`)

Write a function called `is_multimodal()` that detects if the benchmark latencies split into two distinct clusters (e.g., due to CPU throttling or context switching). Steps for implementing it are as follows:

- Check if the number of samples is at least 20. If not, return an **unknown** record stating there are not enough samples.
- Sort the samples and trim the lowest 5% and highest 5% to eliminate extreme outliers.
- Calculate the gaps (differences) between adjacent samples in the trimmed data.
- Compute the median gap. If the median gap is ≤ 0 , return an **unknown** record explaining that the timer resolution is too coarse.
- Find the widest gap in the data and calculate the ratio: `ratio = widest_gap/median_gap`.
- Determine the split point where this widest gap occurs. Count how many samples fall to the left and to the right of this split.
- Return a **measured** record. Set the main boolean value to **True** if the ratio is ≥ 20.0 and both sides contain at least 10% of the total samples; otherwise set it to **False**. Include the gap details and subgroup statistics.

Problem 5: Power Mode and Clock Verification (probe_power_state)

Write a function called `probe_power_state()` that checks the system power mode and verifies if CPU frequencies are fixed. Steps for implementing it are as follows:

- Run the command `nvpmodel -q` using `bench.runner`.
- If the command fails or returns a non-zero exit code, return an `unknown` record explaining the error (e.g., missing sudo permissions).
- If successful, parse the output lines to extract the power mode name from the line containing "NV Power Mode:" and the mode ID number from the line immediately following it.
- Read the minimum CPU frequency from `scaling_min_freq` and the maximum from `scaling_max_freq` using `read_text`.
- Add a key "jetson_clocks" set to `True` if the minimum and maximum frequencies are identical (clocks are locked), `False` if they differ, or `None` if either file could not be read.
- Return the final findings dictionary.

Problem 6: System Hardware Telemetry (probe_telemetry)

Write a function called `probe_telemetry()` that reads temperatures, board power consumption, and GPU utilization from Linux `sysfs`. Steps for implementing it are as follows:

- **Temperature:** Read all available thermal zones under `sys/devices/virtual/thermal`. Convert raw values from millidegrees Celsius to degrees Celsius by dividing by 1000.0. Ignore disabled sensors that report values ≤ -1000 . Report the highest temperature across all valid zones.
- **Power Draw:** Use `read_first` to inspect candidate INA3221 sensor file paths. Parse the raw input as an integer representing power in milliwatts (`power_mw`). Return an `unknown` record if none of the candidate paths exist.
- **GPU Load:** Use `read_first` to check the GPU load candidate files. Convert the raw integer (reported in tenths of a percent, or per-mille) to a percentage by dividing by 10.0. Return an `unknown` record if no GPU load file is found.
- Return a dictionary containing the temperature, power, and GPU utilization records.